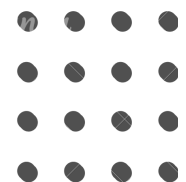




VMES Grinds

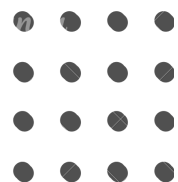




VMES Grinds

Pilot Program - Week 3 - Day 1

Sliding window • Two pointers





Leetcode: 2760
Longest Even Odd Subarray With
Threshold



Moe Myint Mo San



2760. Longest Even Odd Subarray With Threshold

Solved

Easy

Topics

Companies

Hint

You are given a **0-indexed** integer array `nums` and an integer `threshold`.

Find the length of the **longest subarray** of `nums` starting at index `l` and ending at index `r` ($0 \leq l \leq r < \text{nums.length}$) that satisfies the following conditions:

- `nums[l] % 2 == 0`
- For all indices `i` in the range `[l, r - 1]`, `nums[i] % 2 != nums[i + 1] % 2`
- For all indices `i` in the range `[l, r]`, `nums[i] <= threshold`

Return an integer denoting the length of the longest such subarray.

Note: A **subarray** is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [3,2,5,4]`, `threshold = 5`

Output: 3

Explanation: In this example, we can select the subarray that starts at `l = 1` and ends at `r = 3` $\Rightarrow$ `[2,5,4]`. This subarray satisfies the conditions.

Hence, the answer is the length of the subarray, 3. We can show that 3 is the maximum possible achievable length.



Example 2:

Input: `nums = [1,2]`, `threshold = 2`

Output: 1

Explanation: In this example, we can select the subarray that starts at $l = 1$ and ends at $r = 1 \Rightarrow [2]$.

It satisfies all the conditions and we can show that 1 is the maximum possible achievable length.

Example 3:

Input: `nums = [2,3,4,5]`, `threshold = 4`

Output: 3

Explanation: In this example, we can select the subarray that starts at $l = 0$ and ends at $r = 2 \Rightarrow [2,3,4]$.

It satisfies all the conditions.

Hence, the answer is the length of the subarray, 3. We can show that 3 is the maximum possible achievable length.

Constraints:

- `1 <= nums.length <= 100`
- `1 <= nums[i] <= 100`
- `1 <= threshold <= 100`



Problem Explanation Thought Process





Initial Thought Process

To be valid:

- First element must be even
- Each next element must flip parity
- All values must be $\leq$ threshold

This is an alternating pattern detection problem.

A break occurs when:

- threshold is exceeded
- parity repeats

Goal: scan for valid starts $\rightarrow$ extend $\rightarrow$ track maximum.



Algorithm Evolution Overview



Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Try every start, build subarray
Optimal	$O(n)$	$O(1)$	Smart skipping, one-pass scan



Brute Force Approach





Idea:

For each index i:

1. Skip if:

- odd
- or $>$ threshold

2. Otherwise:

- create a temporary subarray
- append starting element
- extend to the right while:
- ✓ $\leq$ threshold
- ✓ parity alternates

Why It Works

We simulate every valid subarray start.



- Uses a temporary list to check parity with last element
- Conceptually simple
- Rechecks parity repeatedly $\rightarrow O(n^2)$

```
class Solution(object):
    def longestAlternatingSubarray(self, nums, threshold):
        ans = 0
        n = len(nums)
        for i in range(n):
            subarray = []
            if nums[i] % 2 == 1 or nums[i] > threshold:
                continue
            subarray.append(nums[i])
            ans = max(ans, len(subarray))
            for j in range(i + 1, n):
                if nums[j] > threshold:
                    break
                if nums[j] % 2 == subarray[-1] % 2:
                    break
                subarray.append(nums[j])
                ans = max(ans, len(subarray))
        return ans
```



Time Complexity (Brute Force)

- Each i expands with a full inner loop in the worst case $\rightarrow O(n^2)$
- Only $O(1)$ extra space (the subarray list is at most n , but recreated per iteration) $\Rightarrow$ Overall: $O(n^2)$ time, $O(1)$ space



Optimal Approach (Linear Scan)





Idea:

Instead of trying all i , only start from valid even positions.

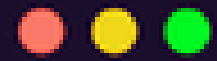
Then expand until:

- threshold breaks
- parity stops alternating

After finishing a valid segment, jump past it:

$$i = j + 1$$

This removes redundant checks and ensures $O(n)$.



```
class Solution:
    def longestAlternatingSubarray(self, nums, threshold):
        n = len(nums)
        ans = 0

        i = 0
        while i < n:
            # Start only if even and <= threshold
            if nums[i] % 2 == 0 and nums[i] <= threshold:
                j = i
                while j + 1 < n and nums[j + 1] <= threshold and (nums[j] % 2 != nums[j + 1] % 2):
                    j += 1
                ans = max(ans, j - i + 1)
                i = j + 1
            else:
                i += 1
        return ans
```




Why This Works

- Starting point is fully determined (must be even + valid)
- ✓ Alternating parity is a strict condition — once broken, cannot continue
- ✓ Using $i = j + 1$ avoids reprocessing overlapping ranges
- ✓ Every element visited at most twice $\rightarrow O(n)$

This is the standard alternating-pattern sliding window.

Time Complexity (Brute Force)

- i jumps over entire alternating segments
- j expands only once per segment
- Each index is visited at most twice $\rightarrow O(n) \Rightarrow$ Overall: $O(n)$ time, $O(1)$ space



Extra Study Material –Visualizations–





Complexity Comparison





Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Rebuilds parity repeatedly
Optimal Scan	$O(n)$	$O(1)$	Smart jumping → linear time



Key Takeaway





This is fundamentally a pattern scanning problem:

- Valid start is deterministic
- Alternation enforces strict constraints
- Threshold acts as a hard barrier
- Linear scan + jumping leads to optimal solution

Recognize pattern → expand → jump → repeat.



Thank You !

